

Object Oriented Systems (OOS) — CO5 Complete Detailed Solutions

Question 5 — Singleton Design Pattern OR Iterator Design Pattern

Based on the uploaded university examination paper.

Introduction

Design Patterns are reusable software design solutions used to solve commonly occurring problems in software engineering. They are not complete programs. Instead, they are:

-  proven design techniques
-  reusable architectural templates
-  object-oriented best practices

Design patterns help developers create software that is scalable, maintainable, reusable, loosely coupled, and extensible .

The famous “Gang of Four (GoF)” categorized design patterns into:

Category	Purpose
Creational	Object creation
Structural	Object composition
Behavioral	Communication and interaction

In this question, two very important patterns are discussed:

-  **Singleton Design Pattern**
-  **Iterator Design Pattern**

Both patterns are extensively used in real-world enterprise systems.  

Question 5(a)

Singleton Design Pattern

Exact Question

Discuss Singleton Design Pattern.

Introduction to Singleton Pattern

The Singleton Design Pattern belongs to the  **Creational Design Pattern Category**. It ensures that:

-  only one object of a class exists
-  a global access point is provided

💡 Why Singleton Exists

In many applications, creating multiple objects can create problems. If multiple objects are created unnecessarily, memory wastage occurs, inconsistent states appear, and synchronization problems arise ⚠️. The Singleton pattern solves this issue elegantly.

Examples:

Real-World Resource	Why Single Instance Needed
Database Connection Manager	Avoid duplicate connections
Printer Spooler	Coordinate print jobs
Configuration Manager	Maintain single configuration
Logger	Centralized logging
Cache Manager	Shared cache access

📘 Core Idea of Singleton Pattern

The Singleton Pattern restricts object creation by:

1. making the constructor private
2. creating a static instance
3. providing a controlled global access method

📘 Structure of Singleton Pattern

```
-----  
| Singleton |  
-----  
| - instance : Singleton |  
-----  
| - Singleton()  
| + getInstance() : Singleton |  
-----
```

💡 Internal Working

🚀 Step-by-Step Execution

1. **Constructor Becomes Private:** `private Singleton()` prevents external object creation using `new Singleton()`.
2. **Static Instance Is Created:** `private static Singleton obj;` Since static belongs to the class, only one copy exists in JVM memory.
3. **Global Access Method Provided:** `getInstance()` returns the same object repeatedly.

📘 Java Program — Singleton Pattern

```
class Singleton {
    private static Singleton obj;

    private Singleton() {
        System.out.println("Singleton Object Created");
    }

    static Singleton getInstance() {
        if(obj == null) {
            obj = new Singleton();
        }
        return obj;
    }

    void display() {
        System.out.println("Inside Singleton Class");
    }
}

class Demo {
    public static void main(String args[]) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        s1.display();
        System.out.println(s1 == s2);
    }
}
```

Output

```
Singleton Object Created
Inside Singleton Class
true
```

Why This Output Occurs

- 🔥 **First Call:** `Singleton.getInstance()` -> Since object is null, JVM creates the object.
- 🔥 **Second Call:** Object already exists. Thus the same object reference returns.
- 🔥 **Comparison:** `s1 == s2` returns `true` because both references point to the same heap object.

JVM Memory Analysis

When a singleton object is created:

- ☒ one heap object allocated
- ☒ static reference stored in method area/class area
- ☒ same reference reused repeatedly Thus memory efficiency improves.

🌟 Types of Singleton Pattern

🚀 **1 Eager Initialization Singleton** Object created immediately during class loading.

```
class Singleton {  
    private static Singleton obj = new Singleton();  
    private Singleton() { }  
    static Singleton getInstance() { return obj; }  
}
```

- **Advantage:** ✅ Thread-safe
- **Disadvantage:** ❌ object created even if unused

🚀 **2 Lazy Initialization Singleton** Object created only when needed.

```
if(obj == null) {  
    obj = new Singleton();  
}
```

- **Advantage:** ✅ memory efficient
- **Disadvantage:** ❌ thread safety issue

🚀 **3 Thread-Safe Singleton**

```
synchronized static Singleton getInstance()
```

- **Why Needed:** In multithreading, two threads may simultaneously create objects. Synchronization prevents duplicate creation.

📋 Advantages of Singleton Pattern

Advantage	Explanation
Controlled Access	Only one instance exists
Memory Efficient	Avoids duplicate objects
Global Access	Accessible everywhere
Consistent State	Shared centralized data

⚠️ Disadvantages of Singleton Pattern

Disadvantage	Explanation
Difficult Testing	Global state complicates unit testing

Disadvantage	Explanation
Tight Coupling	Components depend on singleton
Concurrency Complexity	Thread safety required

💡 **Real-World Intuition** Imagine a country's Prime Minister. In At a given time, only one Prime Minister exists and everyone accesses the same authority. Singleton behaves similarly.

🔗 **Conclusion** Singleton Pattern ensures controlled object creation and centralized access to shared resources. It is one of the most important creational design patterns used heavily in enterprise Java applications.

🚀 Question 5(b)

📘 Iterator Design Pattern

📘 **Exact Question**

Explain Iterator Design Pattern.

🌟 Introduction to Iterator Pattern

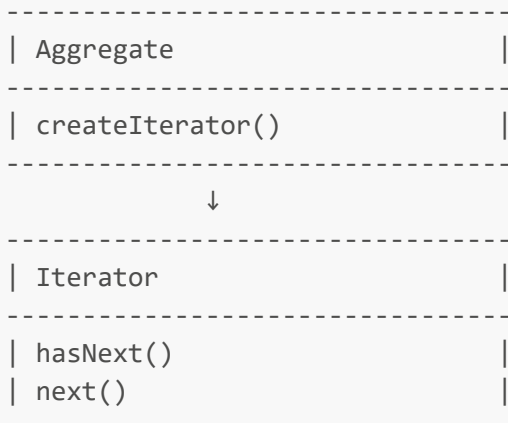
Iterator Pattern belongs to the 🔥 **Behavioral Design Pattern Category**. It provides a way to sequentially access collection elements without exposing internal structure 🚀.

💡 Why Iterator Pattern Exists

Collections may internally use arrays, linked lists, trees, or hash tables. Direct traversal logic becomes complex. Iterator pattern hides complexity and provides a uniform traversal mechanism, encapsulation, and abstraction.

📘 **Real-World Analogy** Imagine a TV remote. 📺 You press "next" or "previous" without knowing the internal electronic circuitry. Iterator works similarly.

📘 Structure of Iterator Pattern



🗨 Components of Iterator Pattern

Component	Purpose
Iterator	Traverses collection
ConcreteIterator	Actual traversal implementation
Aggregate	Collection interface
ConcreteAggregate	Real collection

💻 Java Program — Iterator Pattern

```
import java.util.*;

class Demo {
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("C++");
        list.add("JavaScript");

        Iterator<String> it = list.iterator();

        while(it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

💻 Output

```
Java
Python
C++
JavaScript
```

🗨 Why This Output Occurs

- 🔥 `hasNext()`: Checks whether more elements exist.
- 🔥 `next()`: Returns the next collection element.

💻 Internal JVM Working

Iterator maintains an internal cursor position and the current traversal state. Each `next()` call advances the cursor automatically.

🌟 Manual Custom Iterator Example

```
class Names {
    String arr[] = {"Amit", "Rahul", "Sneha"};
    int index = 0;

    boolean hasNext() {
        return index < arr.length;
    }

    String next() {
        return arr[index++];
    }
}

class Demo {
    public static void main(String args[]) {
        Names n = new Names();

        while(n.hasNext()) {
            System.out.println(n.next());
        }
    }
}
```

Output

```
Amit
Rahul
Sneha
```

Advantages of Iterator Pattern

Advantage	Explanation
Uniform Traversal	Same logic for different collections
Encapsulation	Internal structure hidden
Simplicity	Easier traversal
Flexibility	Multiple traversal strategies possible

Disadvantages of Iterator Pattern

Disadvantage	Explanation
Extra Object Creation	Iterator objects needed
Slight Overhead	Additional abstraction layer
Limited Modification	Concurrent modification issues

Iterator vs Traditional Loop

Feature	Iterator	Traditional Loop
Structure Awareness	Not required	Required
Flexibility	Higher	Lower
Encapsulation	Better	Poor
Collection Compatibility	Universal	Limited

 **Real-World Intuition** Iterator acts like a tour guide  . You simply move through locations one-by-one without worrying about underlying navigation paths.

Final Conclusion

Question 5 discusses two extremely important design patterns: Singleton Pattern and Iterator Pattern.

- **Singleton** focuses on controlled object creation.
- **Iterator** focuses on controlled collection traversal.

Both patterns improve software maintainability, reusability, modularity, and scalability. These design patterns form a critical part of enterprise-level object-oriented software engineering and Java.